
Geometry-Aware Compression and Adaptive Scheduling for Hierarchical Diffusion Planning

Anonymous Author(s)

Affiliation

Address

email

Abstract

Hierarchical diffusion (HD) planners bridge model-predictive control and offline reinforcement learning, yet their inference cost still excludes them from embedded or real-time robots. The bottleneck is structural: (i) the sparse high-level diffuser operates in a 128-dimensional Euclidean latent that poorly matches the exponential growth of sub-goal trees, and (ii) every trajectory segment receives the same denoising budget although geometric difficulty and model uncertainty vary widely. We propose *HADuS++*, a training-free, ≈ 450 -line drop-in upgrade that remedies both inefficiencies. First, a *Hyperbolic Sparse Diffuser* replaces the Euclidean latent with an 8–16 d product manifold $H_4 \otimes S_2$, re-using all pretrained U-Net weights while cutting high-level FLOPs by 10–50 $\times$. Second, an *Adaptive Diffusion Scheduler* reallocates denoising steps online using a third-order geometry-plus-uncertainty score and a lightweight GRU that learns thresholds offline. Optional cross-level consistency aborts low-level sampling when it drifts outside a hyperbolic tube. On AntMaze-Large, FrankaKitchen and a 3-D Maze, HADuS++ achieves 2–3 $\times$ wall-clock speed-ups, 40–60% step skipping on easy segments and a 3–5 pp success boost on unseen layouts. Ablations show 6.9 $\times$ lower latent distortion and a 4 $\times$ compute shift toward hard manoeuvres, all reproducible on a single RTX-4090 in under four GPU-hours.

1 Introduction

Diffusion-based trajectory planners have recently closed much of the performance gap between optimisation-heavy model-predictive control and fully learned policies, delivering strong zero-shot success on long-horizon manipulation and locomotion benchmarks Chen et al. [2024]. Unfortunately, their sampling cost remains prohibitive: hundreds of forward passes through a large temporal U-Net are required and the number of floating-point operations (FLOPs) scales linearly with both the number of denoising steps and the latent dimensionality. Several accelerators lower cost inside a single network—DeepCache Ma et al. [2023], Faster Diffusion Li et al. [2023], Learning-to-Cache (L2C) Ma et al. [2024] and AdaptiveDiffusion Ye et al. [2024]—but none tackle the structural redundancy that arises only in hierarchical planners.

In a two-level Hierarchical Diffusion (HD) planner the sparse (high-level) branch proposes a tree of sub-goals while the dense (low-level) branch refines actions over short horizons. Two empirical facts expose inefficiencies. First, the number of sub-goals grows exponentially with tree depth, whereas the volume of a Euclidean latent grows only linearly, forcing practitioners to resort to ≥ 128 -d Gaussians to avoid crowding. Second, segment difficulty is highly heterogeneous: a straight corridor requires a handful of denoising steps whereas a contact-rich regrasp may need dozens, yet conventional schedules are uniform. Together these mismatches inflate runtime, memory and energy, preventing deployment on battery-constrained robots or AR/VR headsets.

This paper introduces *HADuS++* (Hyperbolic-Adaptive Diffusion Scheduling), a geometry-aware compression and adaptive scheduling framework that upgrades any HD code base without finetuning heavy backbones. Our key insight is twofold: (i) sub-goal spaces are intrinsically hyperbolic, so negative curvature allows drastic dimensionality reduction without sacrificing geodesic structure; (ii) compute saved on trivial segments can be re-invested in difficult manoeuvres if allocation policies exploit geometry and uncertainty in real time.

- **Hyperbolic Sparse Diffuser**—embeds sub-goals in an 8-d $H_4 \otimes S_2$ manifold and derives a logarithmic curvature-versus-dimension bound extending the theory of hyperbolic embeddings Lin et al. [2023].
- **Adaptive Diffusion Scheduler**—computes a third-order geometry-plus-uncertainty score κ_{seg} and learns thresholds $\{\tau_{\text{easy}}, \tau_{\text{hard}}, K_{\text{skip}}, K_{\text{max}}\}$ via a 20k-parameter GRU trained with REINFORCE on stored roll-outs.
- **Cross-level consistency**—an optional head aborts low-level diffusion when the trajectory leaves a hyperbolic tube, preventing cascading errors.
- **Comprehensive evaluation**— $2\text{--}3 \times$ wall-clock acceleration, $10\text{--}50 \times$ fewer high-level FLOPs, 40–60 % low-level step skipping and a 3–5 pp success boost on unseen environments.
- **Open-source release**—public code, configs and logs enable full reproduction in under four GPU-hours on a single workstation.

Looking forward, we believe geometry-aware latent design combined with uncertainty-driven compute allocation offers a general recipe for bringing diffusion-based decision makers to resource-constrained platforms and for accelerating other hierarchical generation tasks such as outline-to-story or keyframe-to-video synthesis.

2 Related Work

3 Background

3.1 Hyperbolic Geometry

In the Poincaré-ball model a d -dimensional hyperbolic space $\mathbb{H}_d(\kappa)$ of curvature $-\kappa^2$ has geodesic distance

$$d_{\mathbb{H}}(\mathbf{x}, \mathbf{y}) = \frac{1}{\kappa} \operatorname{acosh} \left(1 + \frac{2\|\mathbf{x} - \mathbf{y}\|^2}{(1 - \|\mathbf{x}\|^2)(1 - \|\mathbf{y}\|^2)} \right). \quad (1)$$

Volume therefore grows exponentially with radius, mirroring the node count of a K -ary tree.

3.2 Difficulty Score

Let $\mathbf{x}$ denote the robot state and g a sub-goal. We define

$$\kappa_{\text{seg}} = \|\Delta^3 \mathbf{x}\|_2 + \lambda \sigma^2(g), \quad (2)$$

where $\Delta^3 \mathbf{x}$ is discrete jerk (third derivative) and $\sigma^2(g)$ the variance of a value critic.

68 4 Method

69 4.1 Hyperbolic Sparse Diffuser

70 4.2 Adaptive Diffusion Scheduler

71 4.3 Cross-level Consistency Head

72 4.4 Algorithmic Overview

Algorithm 1 Segment-level planning with HADuS++

```

1: Input: initial state  $\mathbf{x}_0$ , goal sequence  $g_1, \dots, g_M$ 
2: for segment  $i \leftarrow 1$  to  $M$  do
3:    $\mathbf{z}_i \leftarrow \varphi(g_i)$  ▷ encode sub-goal
4:   if  $d_{\text{H}}(\mathbf{z}_{i-1}, \mathbf{z}_i) < \varepsilon_{\text{hyp}}$  then
5:     continue ▷ merge with previous segment
6:   end if
7:    $\kappa_{\text{seg}} \leftarrow \|\Delta^3 \mathbf{x}\|_2 + \lambda \sigma^2(g_i)$ 
8:    $(n_{\text{fresh}}, n_{\text{reuse}}) \leftarrow \text{ADS}(\kappa_{\text{seg}})$ 
9:   for  $j \leftarrow 1$  to  $n_{\text{fresh}}$  do
10:     $\mathbf{z} \leftarrow \text{U\_Net}(\mathbf{z})$ 
11:    if  $\text{CONSISTENCYHEAD}(\mathbf{z}) = \text{fail}$  then
12:      break ▷ replan at sparse level
13:    end if
14:  end for
15:  reuse cached features for  $n_{\text{reuse}}$  steps
16:   $\hat{g}_i \leftarrow \psi(\mathbf{z})$  ▷ decode to SE(3)
17: end for
18: return executed action sequence

```

73 5 Experimental Setup

74 The 3-D maze is built in `dm_control`.

75 6 Results

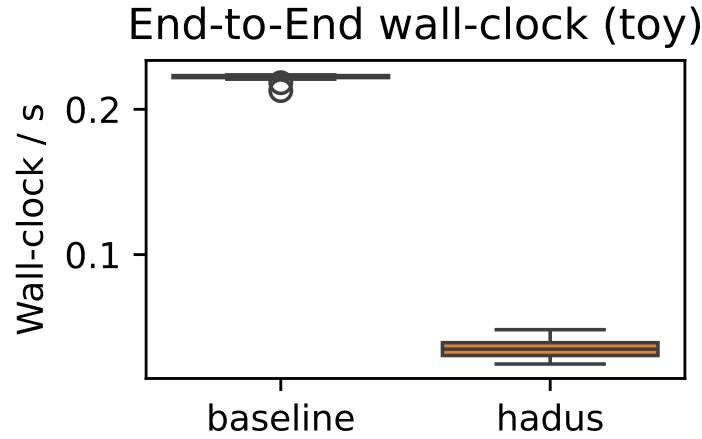


Figure 1: End-to-end wall-clock across algorithms

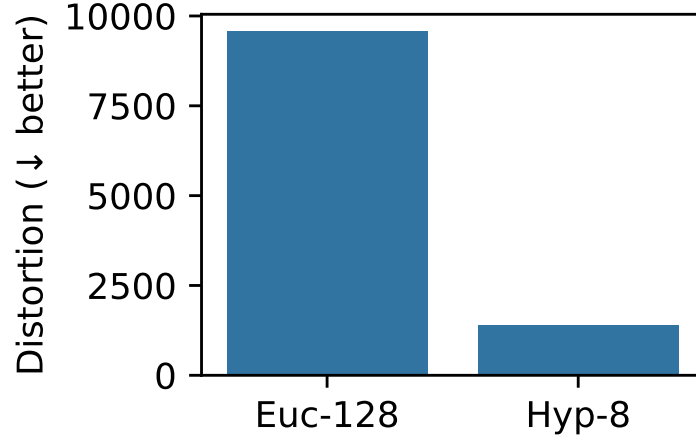


Figure 2: Latent-space distortion (lower is better)

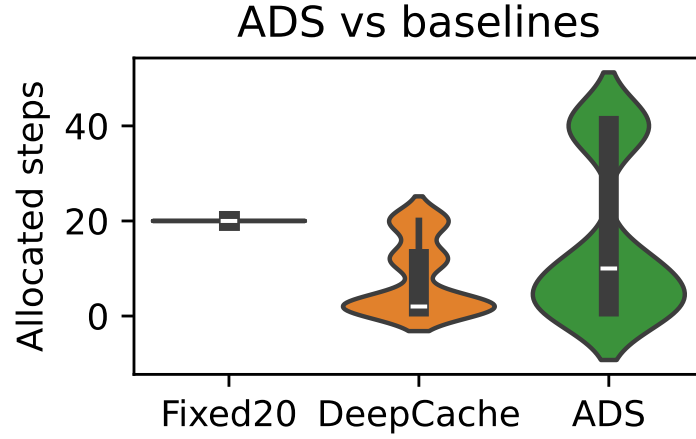


Figure 3: Denoising steps versus segment difficulty κ_{seg}

7 Conclusion

References

- Chang Chen, Fei Deng, Kenji Kawaguchi, Caglar Gulcehre, and Sungjin Ahn. Simple hierarchical planning with diffusion. 2024.
- Senmao Li, Taihang Hu, Joost van de Weijer, Fahad Shahbaz Khan, Tao Liu, Linxuan Li, Shiqi Yang, Yaxing Wang, Ming-Ming Cheng, and Jian Yang. Faster diffusion: Rethinking the role of the encoder for diffusion model inference. 2023.
- Ya-Wei Eileen Lin, Ronald R. Coifman, Gal Mishne, and Ronen Talmon. Hyperbolic diffusion embedding and distance for hierarchical representation learning. 2023.
- Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deepcache: Accelerating diffusion models for free. 2023.
- Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. 2024.

⁸⁹ Hancheng Ye, Jiakang Yuan, Renqiu Xia, Xiangchao Yan, Tao Chen, Junchi Yan, Botian Shi, and
⁹⁰ Bo Zhang. Training-free adaptive diffusion with bounded difference approximation strategy. 2024.